Deep Q-Networks for 10×10 Battleship: Learning to Hunt Without the Rules of Placement

Le Cheng
Khoury College of Computer Sciences
Northeastern University, Boston, MA, USA
cheng.l1@northeastern.edu

***Abstract*—Battleship is a partially observable game in which an agent must sink a hidden fleet in as few shots as possible. Tabular Q-learning fails on it: on a 4×4 board, a Q-table keyed on the visible board plays no better than random, and the classic 10×10 board has more than 3×10^{10} ship layouts. We train Deep Q-Networks (DQN) with a fully convolutional dueling network, Double DQN targets and action masking, using 256 vectorized games to train each agent in about 14 minutes on a laptop. On 4×4, a DQN learned from the raw board reaches 8.92 expected shots, statistically level with the probability-density heuristic (8.86) and within 2% of the exact optimum (8.75). On 10×10, evaluated on 2,000 held-out boards, the board-only DQN reaches 53.0 shots, and adding the density map as an input plane reaches 45.8, 1.2 shots behind probability density (44.6). Using the density as a fixed baseline under a learned correction did not improve on it.**

I. Introduction

In single-player Battleship, ships of lengths 5, 4, 3, 3 and 2 are placed uniformly at random on a 10×10 grid, and each shot reveals *miss*, *hit* or *sunk*. The hidden layout makes this a POMDP [5]. The strongest simple strategy is *probability density* [6]: count every position of every remaining ship consistent with the board, and fire at the cell covered most often. It is greedy and counts ships independently, so in principle a learner that estimates the shots still needed could improve on it.

Our group's tabular agents show why function approximation is needed. A Q-table keyed on the visible 4×4 board visited 3.5 million states in a million games and played at random level (13.9 shots against 14.1). Keying the table on the exact belief (the set of consistent layouts) reached 8.97, but the belief cannot be stored on 10×10, which has 30,093,975,536 layouts. Silver and Veness introduced 10×10 Battleship as a POMDP planning benchmark, solved online with Monte-Carlo tree search over particles [7]; published learning approaches are hobby projects, the best DQN reaching about 52 shots [8]. We ask: *can a DQN [1] learn 10×10 Battleship without being told the placement rules, and does the density map help it as an input feature or as a starting policy?*

II. Method

MDP. The state s is the visible board plus the lengths of the ships still afloat. Observations are exact, so s determines the belief and is a Markov state. Actions are unfired cells. The reward is -1 per shot with $\gamma = 1$, so $Q(s, a)$ is minus the expected number of remaining shots and the return equals minus the game length.

Q-learning and DQN. Q-learning [2] moves $Q(s, a)$ toward the target $y = r + \gamma \max_{a'} Q(s', a')$. DQN replaces the table with a network Q_θ trained on the Huber loss of $Q_\theta(s, a) - y$, sampling transitions from a replay buffer and computing y with a periodically copied target network θ^-. We use Double DQN targets [3]: $y = -1 + Q_{\theta^-}(s', \arg\max_{a'} Q_\theta(s', a'))$, and $y = -1$ when the game ends. Fired cells are masked to $-\infty$ both when acting and inside the target.

Input and network. The board is encoded as planes of 10×10: hit, miss, sunk, an all-ones on-board plane (so zero padding marks the edge), and five planes counting the ships afloat of each length 1–5. The network has four 3×3 convolutions with 32 channels and ReLU, giving a 9×9 receptive field, enough to see a carrier through any cell. A dueling head [4] computes $Q = V(s) + A(s, a) - \overline{A}(s)$: the advantage A comes from a 1×1 convolution (local), and the value V from global mean and max pooling (the shots still needed depend on the whole board). The network has 34.6k weights.

Variants. (A) *board*: the planes above. (B) *density*: plus the probability-density map, normalized so the best cell is 1; it is a function of the board, so it adds no information, only a feature. (C) *residual*: B with $Q = c\,\hat{d}(s, a) + Q_\theta(s, a)$, $c = 1$, and A's last layer initialized to zero, so the untrained agent plays exactly like probability density and DQN learns a correction [9].

Training. 256 games run in parallel as tensors; one iteration fires one shot in every game. Each transition is replayed 8 times on average (batch 256), with a 200k-transition buffer, Adam (learning rate 3×10^{-4}), a target copy every 1,000 steps, and ε decaying linearly from 1.0 (C: 0.1) to 0.02 over the first 10% of 100k games. The games and the density map run on the CPU and the network on the laptop GPU (Apple MPS). Unit tests check that the vectorized games match the reference environment move for move, and that the convolutional density equals the reference heuristic exactly.

III. Experiments

Protocol. Every 5k games we play the greedy policy on 500 validation boards (seeds 100,000+) and keep the best checkpoint. For C the best checkpoint was the untrained one, so we report its final checkpoint. The test set is 2,000 boards (seeds 0–1,999), the same for every agent, played

TABLE I
MEAN SHOTS TO SINK THE FLEET (LOWER IS BETTER)

10×10, 2,000 test boards Agent	Shots	±95%	Paired vs. density	Train
Random	95.27	0.22	+50.62 ± 0.45	–
Hunt-target + parity	50.76	0.39	+6.11 ± 0.55	–
Probability density [6]	44.65	0.40	–	–
Belief greedy (particles)	44.50	0.41	−0.15 ± 0.51	–
DQN-A (board)	53.00	0.45	+8.35 ± 0.54	12.4 min
DQN-B (+ density input)	**45.85**	0.32	+1.20 ± 0.46	14.1 min
DQN-C (residual)	46.05	0.32	+1.41 ± 0.46	14.0 min
4×4, all 264 layouts (exact for deterministic policies)				
Random	14.23			
Q-table on visible board†	13.9			
Q-table on exact belief	8.97			7 min
DQN-A (board)	**8.92**		+0.06 ± 0.26	1.3 min
Probability density	8.86			
Optimal V^* (exact)	8.75			

†Measured by our group on 40 held-out layouts.

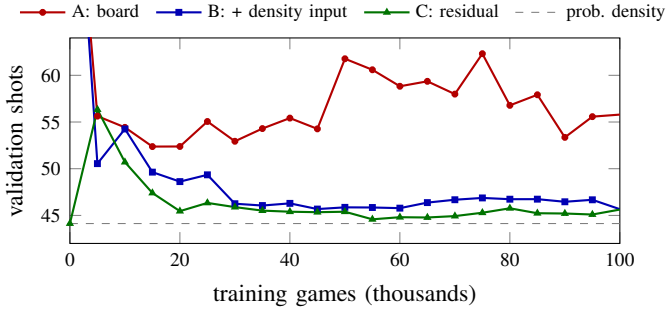


Fig. 1. Greedy validation score during training on 10×10 (500 boards; untrained A and B start at 90.5 and 93.5, off the axis).

through the project's standard engine, which exposes only public information. We report means with 95% intervals and the *paired* difference to probability density on the same boards. On 4×4 we play every one of the 264 layouts, giving the exact expected shots of a deterministic policy.

Results. Table I and Fig. 1 summarize the runs. On 4×4, DQN-A learned from the raw board in 75 seconds and matched probability density, while the board Q-table stayed at random level: the network's generalization across boards is what the table lacked. On 10×10, DQN-A dropped from 90.5 to 52.4 validation shots within 15k games, comparable to the ∼52 reported for an earlier DQN [8], then became unstable (53–62) while its training loss rose steadily from 0.36 to 1.9. Adding the density *input* (B) gained 7.2 shots over A on the test set and came within 1.20 ± 0.46 shots of probability density. The *residual* agent (C) started at the density level, but early updates pushed it to 56 shots; it recovered to about 45 without ever beating its untrained starting point. No learned agent beat probability density; belief greedy, the repo's strongest planner, is statistically level with it (−0.15 ± 0.51).

Where shots are spent. We split each game into shots fired while no unsunk hit is showing (hunting) and while one is (targeting). Probability density uses 27.5 and 17.2 shots. DQN-B uses only 17.1 for hunting but 28.8 while a damaged ship is still afloat: it tends to leave a hit ship and probe elsewhere before finishing it, which costs more than it saves.

IV. DISCUSSION AND CONCLUSION

A convolutional DQN learns Battleship from experience at a fraction of the cost of tabular methods: 4×4 in about a minute and 10×10 in about a quarter of an hour, with no placement rules given. The density map is far more useful as an input feature than as a fixed starting policy: as a feature the network can use it selectively, while as a residual baseline early, poorly calibrated corrections override a near-optimal policy. Greedy play is already within 2% of optimal on 4×4, so the headroom above probability density on 10×10 is likely below one shot, which a learner must capture without first losing more.

Limitations. One training seed per variant; hyperparameters were not tuned; checkpoints were selected on validation boards, disjoint from the test set.

Future work. A lower learning rate and less frequent target copies to remove A's divergence, n-step returns and early stopping to need fewer games, multiple seeds, and a residual agent that learns V before it is allowed to change the density's ranking.

Reproducibility. The code, unit tests, trained models and training logs are in the project repository (package `battleship/dqn`); training and benchmarking each take one command.

ACKNOWLEDGMENT

The code, experiments and analysis were developed with the assistance of Claude (Anthropic), an AI assistant. The author designed the study, directed the work and is responsible for the results. The tabular Q-learning baselines are from the author's project group.

REFERENCES

[1] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.

[3] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI*, 2016.

[4] Z. Wang *et al.*, "Dueling network architectures for deep reinforcement learning," in *Proc. ICML*, 2016.

[5] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, pp. 99–134, 1998.

[6] N. Berry, "Battleship," DataGenetics, Dec. 2011. [Online]. Available: https://www.datagenetics.com/blog/december32011/

[7] D. Silver and J. Veness, "Monte-Carlo planning in large POMDPs," in *Advances in Neural Information Processing Systems 23*, 2010, pp. 2164–2172.

[8] "Deep reinforcement learning–of how to win at Battleship," GA-Intelligence. [Online]. Available: https://www.ga-intelligence.com/deep-reinforcement-learning-win-battleship

[9] T. Johannink *et al.*, "Residual reinforcement learning for robot control," in *Proc. ICRA*, 2019.